DevOps Interview — Complete Q&A Guide

Linux • Git • CI/CD • Docker • Kubernetes • AWS • Terraform

All questions sourced from your screenshots — with detailed answers for each.

Linux & Scripting

Q1. What happens when a Linux server boots?

Answer: The boot process follows BIOS/UEFI → Bootloader (GRUB) → Kernel loading → initramfs → systemd (PID 1) → target units (runlevel). BIOS/UEFI performs hardware POST, GRUB loads the kernel into memory, the kernel initialises hardware and mounts the root filesystem, then systemd starts all services in parallel based on unit dependencies.

Q2. What is the difference between a process and a thread?

Answer: A process is an independent program with its own memory space, file descriptors, and resources. A thread is a lightweight unit of execution within a process — threads share the same memory space. Processes are isolated (one crashing doesn't affect others), while threads communicate faster but a bug in one thread can crash the entire process.

Q3. How do you check CPU, memory, and disk utilization in Linux?

Answer: CPU: 'top' or 'htop' (interactive), 'mpstat -P ALL 1' (per-core stats). Memory: 'free -h' (human-readable), 'vmstat 1' (real-time). Disk: 'df -h' (disk space), 'du -sh /path' (directory size), 'iostat -x 1' (I/O stats). Combined: 'sar' for historical data, 'glances' for an all-in-one view.

Q4. Can you explain the purpose of grep, awk, and sed?

Answer: grep: searches for patterns in text. Example: 'grep -i error /var/log/syslog' finds all lines containing 'error'. awk: processes and transforms structured text/columns. Example: 'awk -F: "{print $1}" /etc/passwd' prints all usernames. sed: stream editor for find-and-replace. Example: 'sed -i s/old/new/g file.txt' replaces all occurrences of 'old' with 'new' in a file.

Q5. What are cron jobs, and when would you use them?

Answer: Cron jobs are scheduled tasks that run automatically at defined intervals using the cron daemon. They follow the format: 'minute hour day month weekday command'. Use cases: automated backups, log rotation, sending reports, database cleanup, health-check pings. Edit with 'crontab -e'. Example: '0 2 * * * /backup.sh' runs backup.sh at 2 AM daily.

Q6. How would you troubleshoot a high CPU utilization issue?

Answer: 1) Run 'top' or 'htop' to identify the PID of the high-CPU process. 2) Use 'ps aux --sort=-%cpu | head' for a snapshot. 3) Run 'strace -p ' to see system calls. 4) Check 'perf top' for kernel-level profiling. 5) Review application logs for infinite loops or abnormal activity. 6) Check for zombie processes with 'ps aux | grep Z'. 7) Use 'nice' or 'renice' to lower process priority temporarily.

Q7. What is the difference between soft links and hard links?

Answer: A hard link is a direct pointer to the same inode (actual data) on disk — deleting the original file doesn't remove the data as long as the hard link exists. A soft link (symlink) is a pointer to the file path — if the original file is deleted, the symlink breaks. Hard links cannot span filesystems or link to directories; symlinks can.

Q8. How do Linux file permissions work?

Answer: Permissions are shown as 'rwxrwxrwx' — three groups of three: owner, group, others. r=read(4), w=write(2), x=execute(1). Example: 'chmod 755 file' = owner has rwx, group and others have r-x. 'chown user:group file' changes ownership. Special bits: setUID (4), setGID (2), sticky bit (1). 'ls -l' shows current permissions.

Q9. What is the difference between Bash and Shell?

Answer: Shell is a generic term for a command-line interpreter (sh, csh, ksh, zsh, bash, etc.). Bash (Bourne Again Shell) is the most widely used shell on Linux — it is a superset of sh with additional features like arrays, arithmetic, command history, tab completion, and advanced scripting capabilities. /bin/sh may point to bash or dash depending on the distro.

Q10. Can you write a shell script to monitor disk usage?

Answer: #!/bin/bash

```
THRESHOLD=80
df -H | grep -vE '^Filesystem|tmpfs|cdrom' | awk '{ print $5 " " $1 }' | while read OUTPUT;
do
USAGE=$(echo $OUTPUT | awk '{ print $1}' | cut -d'%' -f1)
PARTITION=$(echo $OUTPUT | awk '{ print $2 }')
if [ $USAGE -ge $THRESHOLD ]; then
echo "WARNING: Disk usage on $PARTITION is ${USAGE}%"
fi
done
This script checks all mounted partitions and alerts if usage exceeds 80%.
```

Git & Version Control

Q11. What is the difference between Git Merge and Git Rebase?

Answer: 'git merge' combines two branches and creates a new merge commit, preserving full history. 'git rebase' moves your commits on top of another branch, creating a linear history without a merge commit. Use merge for shared/public branches (preserves context). Use rebase for local/feature branches before merging (cleaner history). Never rebase commits already pushed to a shared remote.

Q12. What is a Pull Request, and why is it important?

Answer: A Pull Request (PR) is a request to merge code from one branch into another (typically feature → main). It enables code review — team members can comment, suggest changes, and approve before merging. PRs enforce quality gates (CI checks, test coverage, approval requirements), provide an audit trail of changes, and encourage knowledge sharing across the team.

Q13. How do you resolve merge conflicts in Git?

Answer: 1) Run 'git pull' or 'git merge' — Git marks conflicting files with '<<<<<<<', '=======', '>>>>>>>' markers. 2) Open each conflicting file and manually choose which changes to keep (or combine both). 3) Remove the conflict markers. 4) Stage the resolved files: 'git add '. 5) Complete the merge: 'git commit'. Tools like 'git mergetool' or VS Code's merge editor provide a visual interface.

Q14. What branching strategies have you used in projects?

Answer: Common strategies: (1) Git Flow — main, develop, feature, release, hotfix branches; good for versioned releases. (2) GitHub Flow — main branch always deployable; feature branches merged via PR; ideal for continuous deployment. (3) Trunk-Based Development — all developers commit to main frequently with feature flags; best for CI/CD. Choose based on release cadence and team size.

CI/CD

Q15. What is Continuous Integration (CI)?

Answer: Continuous Integration is the practice of developers merging code changes into a shared repository frequently (multiple times a day). Each merge triggers an automated pipeline that builds the code and runs tests. The goal is to detect integration issues early, reduce merge conflicts, and ensure the codebase is always in a working state. Popular CI tools: Jenkins, GitHub Actions, GitLab CI, CircleCI.

Q16. What is the difference between Continuous Delivery and Continuous Deployment?

Answer: Continuous Delivery (CD): every code change that passes automated tests is automatically prepared and ready to deploy to production — but a human must manually approve and trigger the release. Continuous Deployment: goes one step further — every passing change is automatically deployed to production with no human intervention. Continuous Deployment requires very high test coverage and confidence in automation.

Q17. Can you explain a CI/CD pipeline you have worked on?

Answer: A typical pipeline: (1) Source — developer pushes to GitHub, triggers the pipeline. (2) Build — Maven/npm builds the application. (3) Test — unit tests, integration tests, code coverage checks. (4) Security Scan — SAST tools like SonarQube or Snyk scan for vulnerabilities. (5) Docker Build — containerise the app and push to ECR/DockerHub. (6) Deploy to Staging — automated deployment. (7) Smoke Tests — validate staging. (8) Deploy to Production — with approval gate or automated.

Q18. How would you roll back a failed deployment?

Answer: Strategies: (1) Re-deploy the previous Docker image tag from the container registry. (2) In Kubernetes: 'kubectl rollout undo deployment/' to roll back to the previous ReplicaSet. (3) In CI/CD pipelines (Jenkins/GitHub Actions): re-trigger the last successful pipeline run. (4) For infrastructure: use 'terraform apply' with the previous state. Always store the last 3 working artifact versions. Set up automated rollback triggered by health-check failures.

Docker

Q19. What is containerization, and why is it important?

Answer: Containerization packages an application with all its dependencies (code, runtime, libraries, config) into a single portable unit called a container. Containers run consistently across any environment — dev, staging, production — eliminating the 'it works on my machine' problem. They are lightweight (share the host OS kernel, unlike VMs), start in seconds, and enable microservices architectures where each service runs independently.

Q20. What is the difference between a Docker Image and a Docker Container?

Answer: A Docker Image is a read-only blueprint — a layered filesystem built from a Dockerfile. It is the 'class' in OOP terms. A Docker Container is a running instance of an image — it adds a writable layer on top. You can run multiple containers from the same image. Images are stored in registries (Docker Hub, ECR); containers run on Docker hosts. 'docker build' creates images; 'docker run' creates containers.

Q21. How do Dockerfile layers work?

Answer: Each instruction in a Dockerfile (FROM, RUN, COPY, etc.) creates a new immutable layer. Layers are cached — if a layer hasn't changed, Docker reuses the cache, making builds faster. Layers are stacked to form the final image. Best practice: put frequently changing instructions (COPY source code) at the bottom and stable instructions (RUN apt-get install) at the top to maximise cache hits. Use multi-stage builds to discard build-time layers from the final image.

Q22. What is Docker Compose used for?

Answer: Docker Compose is a tool for defining and running multi-container applications using a YAML file (docker-compose.yml). It lets you define services, networks, and volumes in one file and start everything with 'docker-compose up'. Ideal for local development — for example, spinning up a web app + PostgreSQL + Redis together. Not recommended for production (use Kubernetes or ECS instead). Key commands: 'docker-compose up -d', 'docker-compose down', 'docker-compose logs'.

Q23. How can you reduce Docker image size?

Answer: 1) Use a minimal base image (alpine ~5MB vs ubuntu ~70MB, or distroless). 2) Use multi-stage builds — compile in a builder stage, copy only the binary to the final image. 3) Combine RUN commands to reduce layers: 'RUN apt-get update && apt-get install -y pkg && rm -rf /var/lib/apt/lists/*'. 4) Use .dockerignore to exclude node_modules, .git, test files. 5) Remove dev dependencies before the final COPY. 6) Use 'docker image inspect' and 'dive' tool to analyse layer sizes.

Kubernetes

Q24. What is a Pod in Kubernetes?

Answer: A Pod is the smallest deployable unit in Kubernetes. It wraps one or more containers that share the same network namespace (same IP address and port space) and storage volumes. Containers in a Pod communicate via localhost. Pods are ephemeral — if a Pod dies, Kubernetes creates a new one (with a different IP). In practice, you rarely create Pods directly; you use Deployments or StatefulSets to manage them.

Q25. What is the difference between a Deployment and a StatefulSet?

Answer: Deployment: manages stateless applications. Pods are interchangeable — any Pod can be replaced by another identical one. Pods get random names (e.g., app-7d4b9-xkz2p). Ideal for web servers, APIs. StatefulSet: manages stateful applications (databases, Kafka). Each Pod gets a stable, predictable name (e.g., db-0, db-1), stable network identity, and its own PersistentVolumeClaim. Pods are created and deleted in order. Use StatefulSets for anything that needs persistent identity.

Q26. What is a Service in Kubernetes?

Answer: A Service provides a stable network endpoint to access a set of Pods (which have dynamic IPs). Types: ClusterIP (internal only, default), NodePort (exposes on a port on every node), LoadBalancer (creates a cloud load balancer, external access), ExternalName (maps to a DNS name). Services use label selectors to find the right Pods. They enable load balancing and service discovery within the cluster.

Q27. What are ConfigMaps and Secrets?

Answer: ConfigMap: stores non-sensitive configuration data as key-value pairs (e.g., app settings, feature flags). Injected into Pods as environment variables or mounted as files. Secrets: stores sensitive data (passwords, API keys, TLS certs) in base64-encoded format. Treated with more security — access is restricted via RBAC. In production, integrate with external secret managers (AWS Secrets Manager, HashiCorp Vault) instead of storing sensitive data in etcd.

Q28. What happens when a Pod crashes?

Answer: Kubernetes's kubelet detects the crash and restarts the container based on the Pod's restartPolicy (Always, OnFailure, Never). If the Pod keeps crashing, it enters CrashLoopBackOff — Kubernetes adds exponential backoff between restarts (10s, 20s, 40s... up to 5 min). A Deployment controller ensures the desired number of healthy replicas is maintained — it will create new Pods on other nodes if needed. Use 'kubectl describe pod' and 'kubectl logs --previous' to diagnose.

Q29. Why are namespaces used in Kubernetes?

Answer: Namespaces provide logical isolation within a single cluster. Use cases: separate environments (dev, staging, prod) in one cluster; separate teams or projects; apply resource quotas per namespace (limit CPU/memory per team); apply RBAC policies at namespace level. Default namespaces: default, kube-system (cluster components), kube-public. Best practice: never deploy application workloads in kube-system.

Q30. What are taints and tolerations?

Answer: Taints are applied to nodes to repel Pods — they prevent Pods from being scheduled on that node unless the Pod has a matching toleration. Example: taint a GPU node with 'gpu=true:NoSchedule' so only ML workloads (with the matching toleration) can run on it. Tolerations are added to Pod specs to allow them to be scheduled on tainted nodes. Used for dedicated nodes, spot instances, or nodes with special hardware.

Q31. How would you troubleshoot a Pod stuck in a Pending state?

Answer: Run 'kubectl describe pod ' and read the Events section. Common causes: (1) Insufficient resources — no node has enough CPU/memory to schedule the Pod (increase node size or add nodes). (2) NodeSelector or affinity rules that no node satisfies. (3) Taint on all nodes with no matching toleration on the Pod. (4) PersistentVolumeClaim not bound (storage not available). (5) Image pull secret missing. The Events section will clearly state the scheduling failure reason.

AWS & Azure

Q32. What is the difference between EC2 and Lambda?

Answer: EC2 (Elastic Compute Cloud) is a virtual machine — you provision it, manage the OS, install software, and pay for uptime (by the hour/second). You have full control. Lambda is serverless — you upload a function, AWS manages all infrastructure. Lambda runs only when triggered (HTTP request, S3 event, SQS message) and you pay per invocation. Use EC2 for long-running, stateful, or complex workloads. Use Lambda for event-driven, short-lived tasks (under 15 min) where auto-scaling and zero-management matter.

Q33. What is the difference between Security Groups and NACLs?

Answer: Security Groups (SGs): stateful firewall at the instance/ENI level. If you allow inbound traffic, the return traffic is automatically allowed. Rules are 'allow only' — no explicit deny. Applied to EC2, RDS, Lambda in VPC. NACLs (Network Access Control Lists): stateless firewall at the subnet level. Inbound and outbound rules are evaluated independently. Supports both allow and deny rules, processed in order by rule number. Use NACLs as a second layer of defence to block specific IP ranges at subnet level.

Q34. What is an Application Load Balancer, and how does it work?

Answer: An ALB (Layer 7) distributes incoming HTTP/HTTPS traffic across multiple targets (EC2, ECS tasks, Lambda) based on content. It can route based on URL path (/api → backend, /static → different service), hostname, HTTP headers, or query strings. Features: SSL termination, sticky sessions, health checks, WebSocket support, authentication via Cognito. Unlike Classic LB (Layer 4), ALB understands HTTP and enables sophisticated routing for microservices.

Q35. What is IAM, and why is it important?

Answer: IAM (Identity and Access Management) controls who can do what in your AWS account. It manages Users (humans), Groups (collection of users), Roles (assumed by services/EC2/Lambda), and Policies (JSON documents defining permissions). IAM is critical for security: it enforces least-privilege access, enables MFA, provides audit trails via CloudTrail, and prevents unauthorized access to resources. Never use the root account for daily operations — always use IAM users/roles.

Q36. What is Infrastructure as Code (IaC)?

Answer: IaC is the practice of managing and provisioning cloud infrastructure through machine-readable configuration files (code) rather than manual console clicks. Tools: Terraform (multi-cloud, declarative), AWS CloudFormation (AWS-native), AWS CDK (code-first with Python/TypeScript), Ansible (configuration management). Benefits: version control, repeatability, consistency across environments, peer review of infra changes, automated testing, and disaster recovery (recreate infra in minutes).

Q37. Can you explain a cloud migration project you have worked on?

Answer: A typical lift-and-shift migration: (1) Assess — inventory on-prem servers using AWS Migration Hub or discovery agents. (2) Plan — map each server to target AWS service (VM → EC2, on-prem DB → RDS). (3) Migrate — use AWS Application Migration Service (MGN) for server replication, AWS DMS for databases. (4) Test — validate functionality in AWS while on-prem is still live. (5) Cutover — schedule DNS/traffic switch in a maintenance window. (6) Optimise — right-size instances, add auto scaling, adopt managed services.

Terraform & IaC

Q38. What is Terraform State, and why is it important?

Answer: Terraform State is a JSON file (terraform.tfstate) that tracks the current state of all resources Terraform manages. It maps your configuration to real-world resources. Terraform uses it to calculate the diff (plan) between desired and actual state. Without state, Terraform cannot know what already exists. Always store state remotely (S3 + DynamoDB for locking) — never in version control. State enables team collaboration and prevents duplicate resource creation.

Q39. What are Terraform Modules?

Answer: Modules are reusable, self-contained packages of Terraform configuration. They group related resources (e.g., VPC + subnets + route tables) into a single reusable component. You call a module with 'module "name" { source = "./modules/vpc" }' and pass input variables. Modules promote DRY (Don't Repeat Yourself) principles, ensure consistency, and make large infrastructure codebases manageable. The Terraform Registry hosts community modules for common patterns (AWS VPC, EKS, RDS).

Q40. How do you manage secrets in Terraform?

Answer: Never hardcode secrets in .tf files or commit them to git. Options: (1) Environment variables — set TF_VAR_db_password=secret before running terraform. (2) Terraform Cloud/Enterprise — has a built-in encrypted variable store. (3) HashiCorp Vault provider — fetch secrets dynamically from Vault at plan/apply time. (4) AWS Secrets Manager data source — read secrets at runtime. Mark sensitive variables with 'sensitive = true' to prevent them appearing in plan output.

Q41. How do you handle Terraform state file conflicts?

Answer: Enable remote state with locking: store state in S3 and use a DynamoDB table for state locking. When one person runs 'terraform apply', DynamoDB acquires a lock — others see 'Error: state is locked' until the lock is released. This prevents concurrent applies that cause conflicts. If a lock gets stuck (e.g., after a crash), use 'terraform force-unlock ' to release it manually. Never allow two team members to apply from local state simultaneously.